

LOW-LEVEL DESIGN

RAG-Based Customer Support Assistant

Document Type	Low-Level Design (LLD)
Project	RAG Customer Support System with LangGraph & HITL
Version	1.0
Status	Final

TABLE OF CONTENTS

1. Module-Level Design
2. Data Structures
3. LangGraph Workflow Design
4. Conditional Routing Logic
5. HITL Design
6. API / Interface Design
7. Error Handling

1. MODULE-LEVEL DESIGN

■ Document Processing Module

Responsibilities: Load PDF bytes from filesystem path, validate file format, extract raw page text.

Key class: DocumentProcessor

Dependencies: LangChain PyPDFLoader, os, pathlib

Public Methods:

```
load_document(path: str) -> List[Document]
validate_pdf(path: str) -> bool
extract_metadata(doc: Document) -> dict
```

■ Chunking Module

Responsibilities: Split raw document text into overlapping character-level chunks suitable for embedding.

Key class: ChunkingEngine

Strategy: RecursiveCharacterTextSplitter (chunk_size=500, overlap=50)

Public Methods:

```
chunk_documents(docs: List[Document]) -> List[Chunk]
set_strategy(strategy: str, **kwargs) -> None
```

■ Embedding Module

Responsibilities: Convert text strings to dense float vectors.

Key class: EmbeddingService

Supports: OpenAI text-embedding-3-small, HuggingFace SentenceTransformer

Public Methods:

```
embed_texts(texts: List[str]) -> List[List[float]]
embed_query(query: str) -> List[float]
```

■ Vector Storage Module

Responsibilities: Persist embeddings + chunk metadata, support similarity queries.

Key class: VectorStore (wraps chromadb.Client)

Collection schema: id, embedding, document, metadata{source,page,chunk_id}

Public Methods:

```
upsert_chunks(chunks: List[EmbeddedChunk]) -> None
similarity_search(query_vec: List[float], k: int) -> List[SearchResult]
delete_collection(name: str) -> None
```

■ Retrieval Module

Responsibilities: Accept user query, embed it, return top-k ranked context chunks.

Key class: Retriever

Public Methods:

```
retrieve(query: str, k: int = 4) -> List[SearchResult]
get_scores(results: List[SearchResult]) -> List[float]
```

■ Query Processing Module

Responsibilities: Classify intent, build LLM prompt, invoke LLM, parse response.

Key class: QueryProcessor

Public Methods:

```
classify_intent(query: str) -> IntentLabel
build_prompt(query: str, context: List[str]) -> str
invoke_llm(prompt: str) -> LLMResponse
```

■ Graph Execution Module

Responsibilities: Define and compile LangGraph StateGraph, expose run() entrypoint.

Key class: WorkflowGraph

Public Methods:

```
build_graph() -> CompiledGraph
run(query: str) -> GraphState
get_state_snapshot() -> dict
```

■ HITL Module

Responsibilities: Enqueue escalated queries, expose interface for human response, inject human answer back into graph state.

Key class: HITLManager

Public Methods:

```
escalate(state: GraphState) -> EscalationTicket
submit_human_response(ticket_id: str, response: str) -> None
poll_resolved(ticket_id: str) -> Optional[str]
```

2. DATA STRUCTURES

2.1 Document Representation

```
@dataclass
class Document:
    doc_id: str # UUID
    source: str # file path
    page_no: int # 0-indexed page number
    content: str # raw extracted text
    metadata: dict # arbitrary key-value pairs
```

2.2 Chunk Format

```
@dataclass
class Chunk:
    chunk_id: str # UUID
    doc_id: str # parent document UUID
    content: str # chunk text (~500 chars)
    start_idx: int # character offset in source doc
    end_idx: int
    metadata: dict # source, page_no, chunk_index
```

2.3 Embedding Structure

```
@dataclass
class EmbeddedChunk(Chunk):
    embedding: List[float] # 1536-dimensional vector
    model: str # embedding model name
    created_at: datetime
```

2.4 Query-Response Schema

```
@dataclass
class QueryResponse:
    query: str
    answer: str
    sources: List[str] # chunk_ids used
    confidence: float # 0.0 - 1.0
    escalated: bool
    latency_ms: int
    timestamp: datetime
```

2.5 LangGraph State Object

```
class GraphState(TypedDict):
    query: str
    intent: str # 'faq' | 'complaint' | 'escalate'
    retrieved: List[str] # context chunks
    scores: List[float] # similarity scores
    answer: str
    confidence: float
```

```
escalated: bool
ticket_id: Optional[str]
human_response:Optional[str]
error: Optional[str]
```

3. LANGGRAPH WORKFLOW DESIGN

3.1 Nodes

Node Name	Input State Fields	Output State Fields	Responsibility
input_node	query	query, intent	Classify user intent
retrieve_node	query	retrieved, scores	Embed query; fetch top-4 chunks
generate_node	query, retrieved	answer, confidence	Build prompt; call LLM
route_node	confidence, intent	(edge decision)	Conditional branch decision
output_node	answer	answer (final)	Format and return response
escalate_node	query, intent	escalated, ticket_id	Create HITL ticket
hitl_node	ticket_id	human_response	Wait/poll for human response

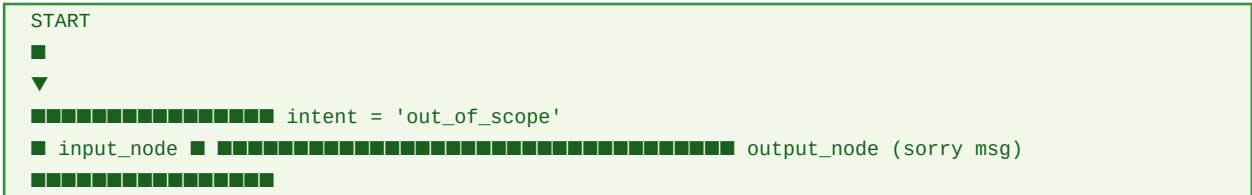
3.2 Edges & Conditional Routing

```
graph = StateGraph(GraphState)
# Add nodes
graph.add_node('input', input_node)
graph.add_node('retrieve', retrieve_node)
graph.add_node('generate', generate_node)
graph.add_node('route', route_node)
graph.add_node('output', output_node)
graph.add_node('escalate', escalate_node)
graph.add_node('hitl', hitl_node)

# Fixed edges
graph.add_edge(START, 'input')
graph.add_edge('input', 'retrieve')
graph.add_edge('retrieve', 'generate')
graph.add_edge('generate', 'route')

# Conditional edge from route_node
graph.add_conditional_edges(
    'route',
    routing_function, # returns 'output' | 'escalate'
    {'output': 'output', 'escalate': 'escalate'}
)
graph.add_edge('escalate', 'hitl')
graph.add_edge('hitl', 'output')
graph.add_edge('output', END)
```

3.3 State Transitions Diagram



[illegible]

4. CONDITIONAL ROUTING LOGIC

4.1 Routing Function

```
def routing_function(state: GraphState) -> str:
    confidence = state['confidence'] # float 0-1
    scores = state['scores'] # list of similarity scores
    answer = state['answer']
    intent = state['intent']
    max_sim = max(scores) if scores else 0.0
    no_context = max_sim < 0.45
    low_confidence= confidence < 0.70
    complex_query = intent == 'complaint' and len(answer) < 80
    uncertain_llm = 'I don't know' in answer or 'unclear' in answer.lower()
    if no_context or low_confidence or complex_query or uncertain_llm:
        return 'escalate'
    return 'output'
```

4.2 Escalation Criteria Summary

Condition	Threshold	Action
Max similarity score	< 0.45	Escalate – knowledge gap
LLM confidence	< 0.70	Escalate – uncertain answer
Intent = complaint	AND answer short	Escalate – needs human empathy
LLM says uncertain	'I don't know'	Escalate – explicit hedge
Out-of-scope intent	Any	Polite out-of-scope message

5. HITL DESIGN

5.1 Escalation Trigger

- Routing function returns 'escalate' for any of the conditions in Section 4.2.
- `escalate_node` creates an `EscalationTicket` with `status=OPEN`, stores in ticket queue.
- User receives immediate acknowledgement: 'Connecting you to a human agent...'

5.2 Human Agent Workflow

```
# Agent dashboard polls open tickets
tickets = hitl_manager.get_open_tickets() # returns List[EscalationTicket]

# Agent reads context and provides response
hitl_manager.submit_human_response(
    ticket_id = ticket.id,
    response = 'Here is the resolution...',
    agent_id = 'agent_007'
)

# Graph state updated; output_node formats and returns response
```

5.3 HITL Data Structure

```
@dataclass
class EscalationTicket:
    ticket_id: str
    query: str
    context: List[str] # retrieved chunks
    reason: str # why escalated
    status: str # OPEN | IN_PROGRESS | RESOLVED
    created_at: datetime
    agent_id: Optional[str]
    human_response: Optional[str]
    resolved_at: Optional[datetime]
```

5.4 Benefits & Limitations

Benefits	Limitations
Handles edge cases beyond LLM capability	Adds latency when agent is unavailable
Builds trust with users for sensitive queries	Requires staffed support queue
Provides training data for future model improvements	Asynchronous — user may have to wait

6. API / INTERFACE DESIGN

6.1 REST Endpoint (FastAPI)

```
POST /api/v1/query
Request body (JSON):
{
  "query": "string", // required
  "session_id": "string", // optional, for conversation context
  "filters": {"source": "str"} // optional metadata filter
}
Response body (JSON):
{
  "answer": "string",
  "sources": ["chunk_id_1", ...],
  "confidence": 0.87,
  "escalated": false,
  "ticket_id": null,
  "latency_ms": 1240
}
```

6.2 CLI Interface

```
# Ingest a PDF
python main.py ingest --path ./docs/faq.pdf

# Run interactive query loop
python main.py query

# Submit human response to open ticket
python main.py hitl --ticket TKT-001 --response 'Here is the answer...'
```

7. ERROR HANDLING

Error Scenario	Detection	Handling Strategy
PDF not found / corrupt	os.path check + try/except	Return user-facing error; log and skip
No chunks after split	len(chunks)==0 guard	Raise ValueError; alert ingestion pipeline
Embedding API failure	HTTP 5xx / timeout catch	Retry x3 with exp. backoff; fallback to local model
ChromaDB unavailable	ConnectionError on upsert/query	Return 503; queue request for retry
No relevant chunks (k=0)	scores all < 0.3	Route to escalation immediately
LLM timeout / error	requests.Timeout, HTTP 5xx	Retry x2; if persists, escalate to HITL
Graph state corruption	KeyError in node function	Catch in graph runner; return safe error response

All errors are logged to a structured JSON log file with timestamp, node name, error type, and stack trace for post-mortem analysis.